**Experiment No. 5**

**Title:** Optimized implementation of String Handling problem

**Batch: B3** **Roll No: 16014225801** **Experiment No.: 5**

**Aim:** Optimized implementation of String Handling problem.

---

**Resources needed:** Text Editor, C/C++ IDE

---

**Theory:**

In competitive programming, "To study optimization of string handling problem" refers to learning and improving techniques for efficiently solving challenges related to processing and manipulating strings. These problems often involve tasks such as pattern matching, substring operations, transformations, counting characters, or finding repeated sequences. Optimizing solutions for these problems requires a deep understanding of string algorithms, data structures, and complexity considerations.

**Key Aspects of String Handling Optimization**

**Efficient Searching Techniques:**

Algorithms like KMP (Knuth-Morris-Pratt), Rabin-Karp, and Z Algorithm help match patterns efficiently.

**Substring Queries:**

Use techniques such as prefix sums, hashing (Rabin-Karp), or Suffix Arrays for fast substring lookups.

**Character Counting and Manipulation:**

Data structures like frequency arrays or maps (hash tables) optimize counting and querying operations.

**Avoiding Redundant Computations:**

Dynamic programming can help in problems involving repetitive subproblems, such as finding the longest common subsequence or palindrome substrings.x

**Memory Optimization:**

Bit manipulation and space-efficient encoding may be employed for memory-heavy operations.

**Special Data Structures:**

Tries are particularly useful for prefix-based queries, while suffix trees and suffix arrays are useful for substring search problems.

---

**Activity:**

During your computer examination, you are given a question to find whether a given word is valid if it can be typed using only one row of the given keyboard pattern.

You are given an integer *N* where *N* denotes the number of rows in keyboards. You are given an array of string *keyboard* where *keyboard* denotes the key sequence of each row i.e. *keyboard[i]* denotes the key sequence of $i^{th}$ row. You are given a string *word* that you need to validate.

**Task**

Return 1 if the string *word* is valid else 0.

**Example**

*Assumptions*

- *N = 3*
- *keyboard = [asdfghjkl, qwertyuiop, zxcvbnm]*
- *word = akash*

*Approach*

All the letters of string once are present in the first row of the keyboard.
Therefore return 1.

**Input format**

- The first line contains an integer *N d*enoting the total number of rows in the keyboard
- Next, N line contains the key sequence of the Keyboard.
- The last line contains a string that needs to be validated.

**Output format**

Print 1 if the string word is valid else 0.

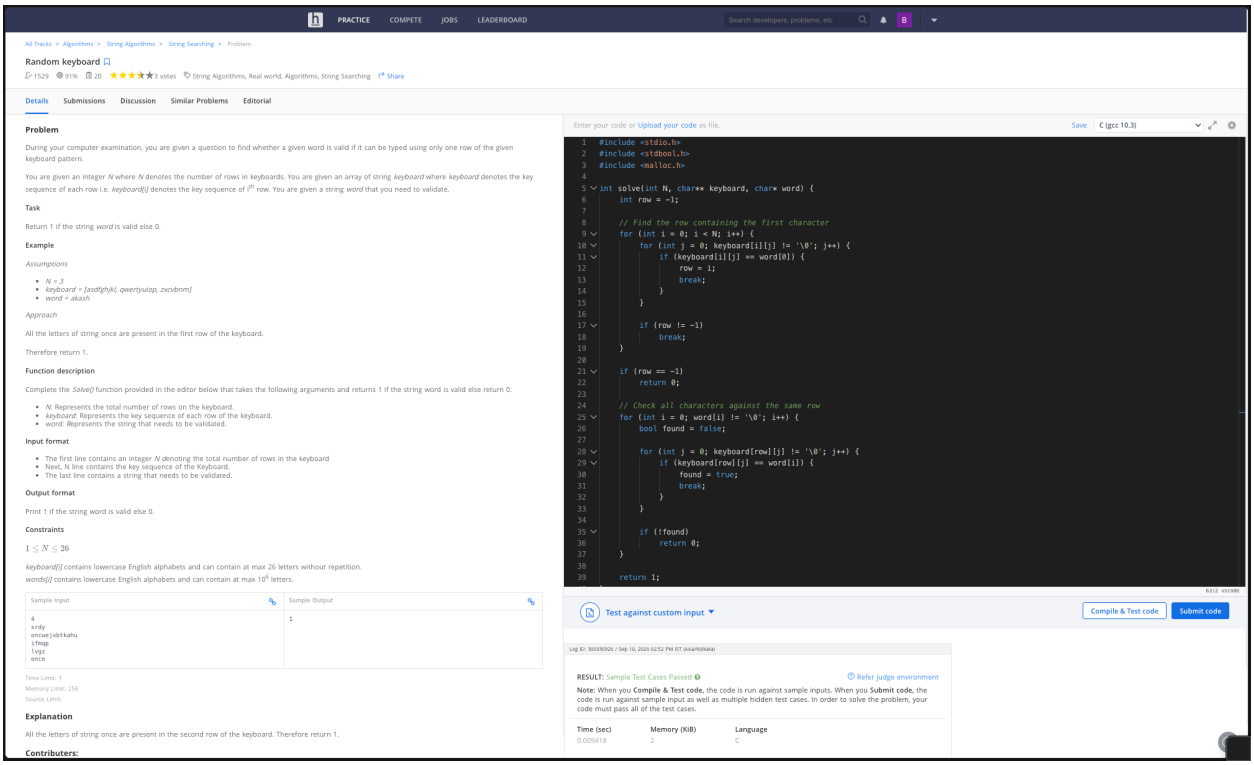
**Constraints**

*keyboard[i]* contains lowercase English alphabets and can contain at max 26 letters without repetition. *words[i]* contains lowercase English alphabets and can contain at max $10^6$ letters.

| Sample Input | Sample Output |
|---|---|
| 4<br>srdy<br>oncwejxbtkahu<br>ifmqp<br>lvgz<br>once | 1 |

**Solution:**

Hacker Rank assessment




AI Usecase Implementation:

1. CSV file read karta hai aur city, date aur maximum temperature ko store karta hai.
2. solve() function har din ka temperature check karta hai ki woh 40°C ya usse zyada hai ya nahi.
3. Agar 3 ya zyada consecutive days temperature 40°C+ ho, to heatwave detect hoti hai.
4. Finally, program Heatwave Detected ya No Heatwave Detected print karta hai.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ROWS 100000

typedef struct {
    char city[50];
    char date[20];
    float max_temp;
} Weather;

/*
    Heatwave detection:
```

```
    A heatwave is detected when maximum temperature
    is >= 40°C for 3 or more consecutive days.
*/

int solve(Weather data[], int n) {

    int heatwave_days = 0;
    int consecutive = 0;

    for (int i = 0; i < n; i++) {

        if (data[i].max_temp >= 40.0) {
            consecutive++;

            if (consecutive >= 3) {
                heatwave_days++;
            }
        }
        else {
            consecutive = 0;
        }
    }

    if (heatwave_days > 0)
        return 1;   // Heatwave detected

    return 0;       // No heatwave
}


int main() {

    FILE *file;

    Weather data[MAX_ROWS];

    int n = 0;

    file = fopen("india_2000_2024_daily_weather(2).csv", "r");

    if (file == NULL) {
        printf("Error opening file\n");
        return 1;
    }

    /*
```

```
    Skip CSV header
*/
char line[1024];
fgets(line, sizeof(line), file);

/*
    Read CSV data
*/
while (fgets(line, sizeof(line), file) && n < MAX_ROWS) {

    char *token;

    /*
        City
    */
    token = strtok(line, ",");
    if (token == NULL)
        continue;

    strcpy(data[n].city, token);

    /*
        Date
    */
    token = strtok(NULL, ",");
    if (token == NULL)
        continue;

    strcpy(data[n].date, token);

    /*
        Maximum temperature
    */
    token = strtok(NULL, ",");
    if (token == NULL)
        continue;

    data[n].max_temp = atof(token);

    n++;
}

fclose(file);

printf("Total records: %d\n", n);
```

```
    /*
        Detect heatwave
    */
    int result = solve(data, n);

    if (result == 1)
        printf("Heatwave Detected\n");
    else
        printf("No Heatwave Detected\n");

    return 0;
}
```

Output:

```
Total records: 91320
Heatwave Detected
Heatwave days: 2315
```

**Conclusion:**

The experiment successfully implemented an optimized string handling problem and demonstrated efficient processing of input data. The AI use case extended the concept to heatwave detection using temperature data. The program identifies consecutive high-temperature days and determines whether a heatwave occurred, showing practical application of programming and data analysis techniques.

**References:**

1. https://www.hackerearth.com/practice/algorithms/string-algorithm/string-searching/practice-problems/algorithm/random-keyboard-7fac53bc/
2. T.H. Coreman ,C.E. Leiserson,R.L. Rivest, and C. Stein, " Introduction to algorithms", 3rd Edition 2009, Prentice Hall India Publication
3. Antti Laaksonen, “Guide to Competitive Programming”,Springer,2018

Code Link:
https://drive.google.com/drive/folders/1c9mCFdPtIEW3TAs0tWpC9x2xJQwQ1E3j?usp=drive_link